MÓDULO 2: APIs (I) 🌐

Extracción de Datos en Tiempo Real y Fuentes Externas

APIs

¿Qué es una API?

Una **API** (*Application Programming Interface*) es un conjunto de reglas y protocolos que permite a dos aplicaciones o sistemas **comunicarse** e intercambiar datos de forma estructurada y segura.

Piensa en ella como un traductor universal que permite que diferentes programas hablen el mismo idioma, sin importar cómo estén construidos internamente.

Las APIs permiten la automatización completa de la obtención de datos y el acceso a información en tiempo real, eliminando la necesidad de descargas manuales o actualizaciones constantes.

El Cliente (Tú)

Tu código Python que necesita información específica

El Camarero (API)

Toma tu pedido y te trae exactamente lo que necesitas

La Cocina (Servidor)

Donde se almacena toda la información disponible

La Estructura de la Petición y la Respuesta

Cada interacción con una API sigue un patrón predecible: tú envías una **solicitud** (request) y recibes una **respuesta** (response). Comprender estos elementos es fundamental para trabajar eficazmente con cualquier API.

Concepto	Rol en la Extracción de Datos	Ejemplo
Endpoint	La **URL específica** que representa un recurso o funcionalidad concreta. Es la dirección exacta a la que enviamos nuestra petición.	https://pokeapi.co/api/v2/pokemon?limit=100000&offset=0
Métodos HTTP	El tipo de **acción** que queremos realizar sobre ese recurso del servidor.	**GET** (Obtener datos), **POST** (Crear datos nuevos)
Request	La **solicitud completa** que enviamos al servidor, incluyendo el método HTTP, el endpoint y cualquier parámetro adicional.	Un requests.get() enviado a la URL del endpoint
Response	La **respuesta** que devuelve el servidor, incluyendo los datos solicitados y un código de estado que indica el resultado.	Los datos del Pokémon en formato JSON con código 200

Dato clave: El método **GET** es el más utilizado en análisis de datos porque nos permite *obtener* información sin modificar nada en el servidor.

Usando requests: El Cliente HTTP de Python

La librería `requests` es la herramienta estándar en Python para realizar peticiones HTTP a APIs. S

Con `requests` podemos comunicarnos con cualquier API que siga los protocolos HTTP estándar, que son la inmensa mayoría.

Ejemplo Práctico: Consultando la PokéAPI

```
import requests

# 1. Importar la librería necesaria

# 2. Hacer la solicitud GET al Endpoint
response = requests.get('https://pokeapi.co/api/v2/pokemon/pikachu')

# 3. Acceder al contenido en formato JSON
datos = response.json()

# 4. Trabajar con los datos como un diccionario
print(f"Nombre: {datos['name']}")
print(f"Altura: {datos['height']}")
print(f"Peso: {datos['weight']}")
```

01

Importar requests

Cargamos la librería en nuestro entorno de trabajo

02

Enviar la petición

Usamos `requests.get()` con el endpoint deseado

03

Convertir a JSON

El método `.json()` transforma la respuesta en un diccionario Python

04

Extraer información

Accedemos a los datos usando claves, como en cualquier diccionario

JSON (JavaScript Object Notation): Es el formato de datos más común en las APIs modernas. En Python, se traduce automáticamente a **diccionarios y listas**, estructuras que ya conoces bien.

Códigos de Estado: ¿Tu Solicitud Tuvo Éxito?

Cada respuesta de una API incluye un **código de estado HTTP** (accesible mediante `response.status_code`) que indica el resultado de tu petición. Comprender estos códigos es esencial para detectar y solucionar problemas en tus extracciones de datos.

2XX - Éxito ✅

200 OK

La petición fue exitosa y los datos están disponibles en el cuerpo de la respuesta. Este es el código que queremos ver siempre.

- Tu solicitud fue correcta
- Los datos están listos para usar
- Todo funcionó según lo esperado

4XX - Error del Cliente ❌

404 Not Found

El recurso o endpoint solicitado no existe. Probablemente hay un error en la URL o el recurso fue eliminado.

401 / 403 No Autorizado

Tienes problemas de autenticación o permisos. Necesitas proporcionar credenciales válidas (API key o token).

- Verifica la URL del endpoint
- Comprueba tus credenciales
- Revisa los permisos de tu cuenta

5XX - Error del Servidor ⚙️

500 Internal Server Error

Algo salió mal del lado del servidor de la API. No es un problema con tu código, sino con el servicio al que intentas acceder.

- El problema está en el servidor, no en tu código
- Intenta de nuevo más tarde
- Contacta al soporte de la API si persiste

```
# Ejemplo: Verificando el código de estado
response = requests.get('https://api.ejemplo.com/datos')

if response.status_code == 200:
 print("
```

Códigos de Estado: ¿Tu Solicitud Tuvo Éxito?

1xx	Informativo	Petición recibida	La petición ha sido recibida y el proceso continúa.
200	Éxito	OK	La petición ha tenido éxito. Los datos solicitados se encuentran en el cuerpo de la respuesta.
201	Éxito	Created	La petición ha tenido éxito y un nuevo recurso ha sido creado.
204	Éxito	No Content	La petición ha tenido éxito, pero no hay contenido que devolver.
301	Redirección	Moved Permanently	El recurso solicitado ha sido movido permanentemente a una nueva URL.
400	Error del Cliente	Bad Request	La solicitud no pudo ser entendida por el servidor debido a una sintaxis incorrecta.
401	Error del Cliente	Unauthorized	La solicitud requiere autenticación. Acceso denegado.
403	Error del Cliente	Forbidden	El servidor entiende la solicitud, pero se niega a autorizarla.
404	Error del Cliente	Not Found	El servidor no pudo encontrar el recurso solicitado.
500	Error del Servidor	Internal Server Error	Ha ocurrido un error inesperado en el servidor.
503	Error del Servidor	Service Unavailable	El servidor no está listo para manejar la solicitud (ej. mantenimiento).

Resumen

API

El **conector** que te da acceso estructurado y programático a datos externos en tiempo real

requests

La **herramienta** en Python para enviar solicitudes HTTP (GET, POST) a cualquier API

Endpoint

La **URL específica** que identifica el recurso exacto al que quieres acceder

response.json()

El método que **convierte** la respuesta en diccionarios y listas de Python (formato JSON)

status_code

La forma de **verificar** si tu extracción fue exitosa (200 = éxito)